

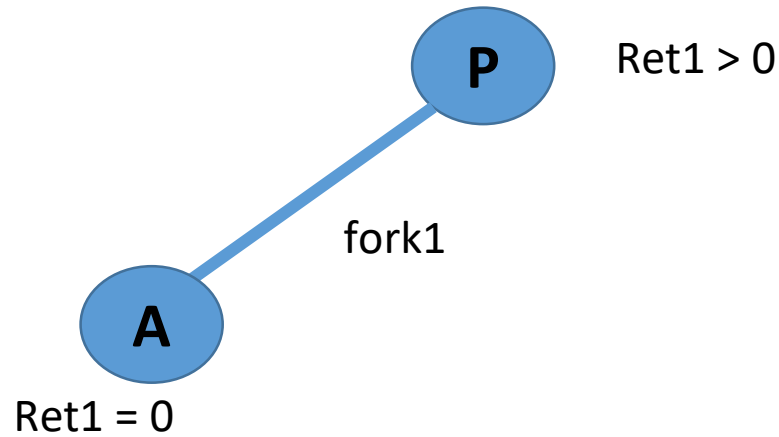
Operating Systems

Fork and Process Tree Animation

Azizur Rahman Anik
Lecturer, Dept of CSE, UIU

Process P

```
1. pid_t ret1 = fork();
2. pid_t ret2 = fork();
3.
4. if(ret2 == 0)
5. {
6.     printf("OS is the worst!\n");
7. }else if(ret1 == 0 && ret2 > 0)
8. {
9.     wait(NULL);
10.    printf("OS is the okay!\n");
11. }else if(ret1 > 0 && ret2 == 0)
12. {
13.     wait(NULL);
14.     printf("OS is fun!\n");
15. }else if(ret1 > 0 && ret2 > 0)
16. {
17.     int status;
18.     waitpid(ret2, &status, 0);
19.     printf("OS is confusing!\n");
20. }else
21. {
22.     wait(NULL);
23.     printf("OS is the best!\n");
24. }
```



Now the system has two processes
and instruction pointer is at line 2
for each process!

Process A

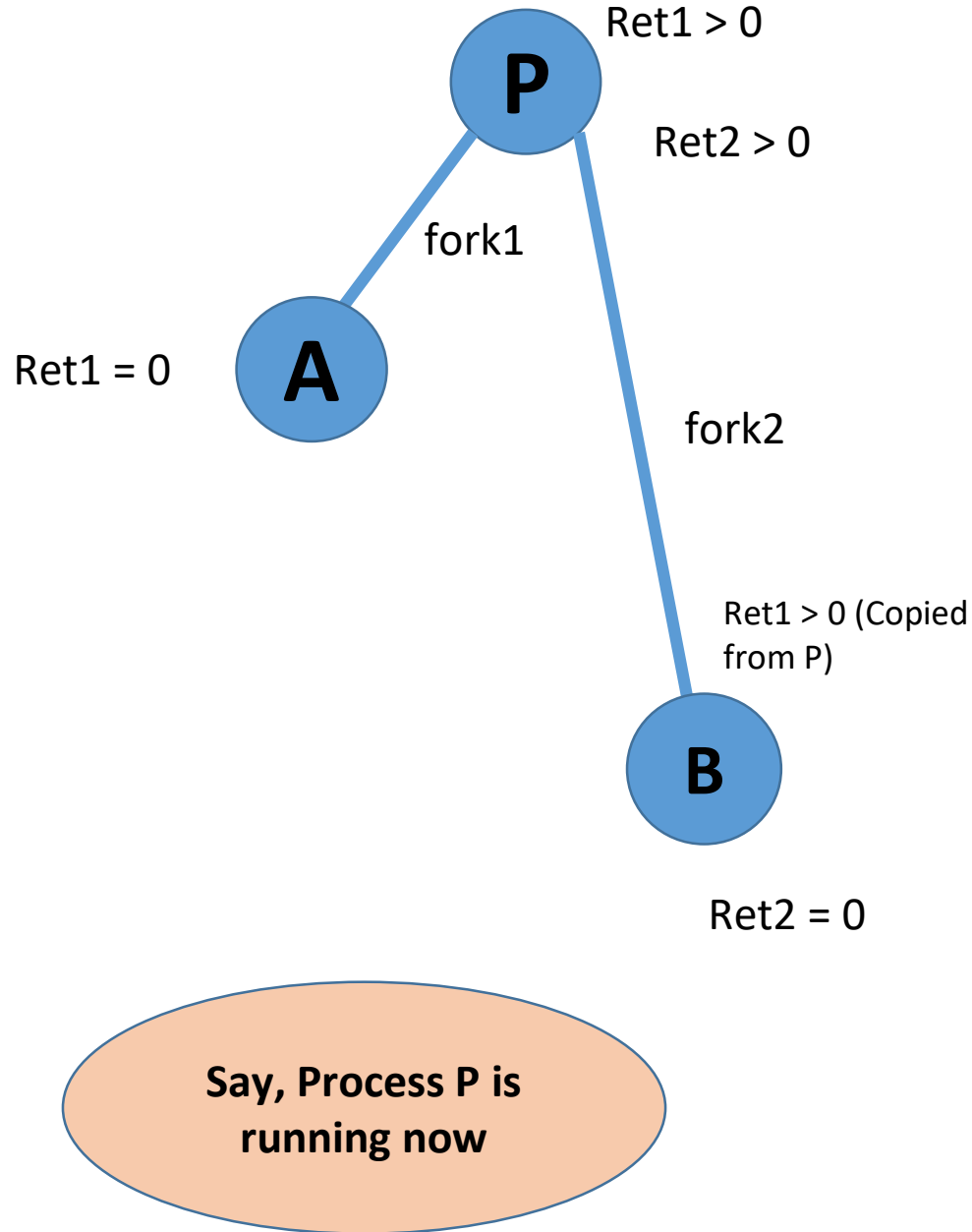
```
1. pid_t ret1 = fork();
2. pid_t ret2 = fork();
3.
4. if(ret2 == 0)
5. {
6.     printf("OS is the worst!\n");
7. }else if(ret1 == 0 && ret2 > 0)
8. {
9.     wait(NULL);
10.    printf("OS is the okay!\n");
11. }else if(ret1 > 0 && ret2 == 0)
12. {
13.     wait(NULL);
14.     printf("OS is fun!\n");
15. }else if(ret1 > 0 && ret2 > 0)
16. {
17.     int status;
18.     waitpid(ret2, &status, 0);
19.     printf("OS is confusing!\n");
20. }else
21. {
22.     wait(NULL);
23.     printf("OS is the best!\n");
24. }
```

Process P

```

1. pid_t ret1 = fork();
2. pid_t ret2 = fork();
3.
4. if(ret2 == 0)
5. {
6.     printf("OS is the worst!\n");
7. }else if(ret1 == 0 && ret2 > 0)
8. {
9.     wait(NULL);
10.    printf("OS is the okay!\n");
11. }else if(ret1 > 0 && ret2 == 0)
12. {
13.     wait(NULL);
14.    printf("OS is fun!\n");
15. }else if(ret1 > 0 && ret2 > 0)
16. {
17.     int status;
18.     waitpid(ret2, &status, 0);
19.     printf("OS is confusing!\n");
20. }else
21. {
22.     wait(NULL);
23.     printf("OS is the best!\n");
24. }

```



Process B

```

1. pid_t ret1 = fork();
2. pid_t ret2 = fork();
3.
4. if(ret2 == 0)
5. {
6.     printf("OS is the worst!\n");
7. }else if(ret1 == 0 && ret2 > 0)
8. {
9.     wait(NULL);
10.    printf("OS is the okay!\n");
11. }else if(ret1 > 0 && ret2 == 0)
12. {
13.     wait(NULL);
14.    printf("OS is fun!\n");
15. }else if(ret1 > 0 && ret2 > 0)
16. {
17.     int status;
18.     waitpid(ret2, &status, 0);
19.     printf("OS is confusing!\n");
20. }else
21. {
22.     wait(NULL);
23.     printf("OS is the best!\n");
24. }

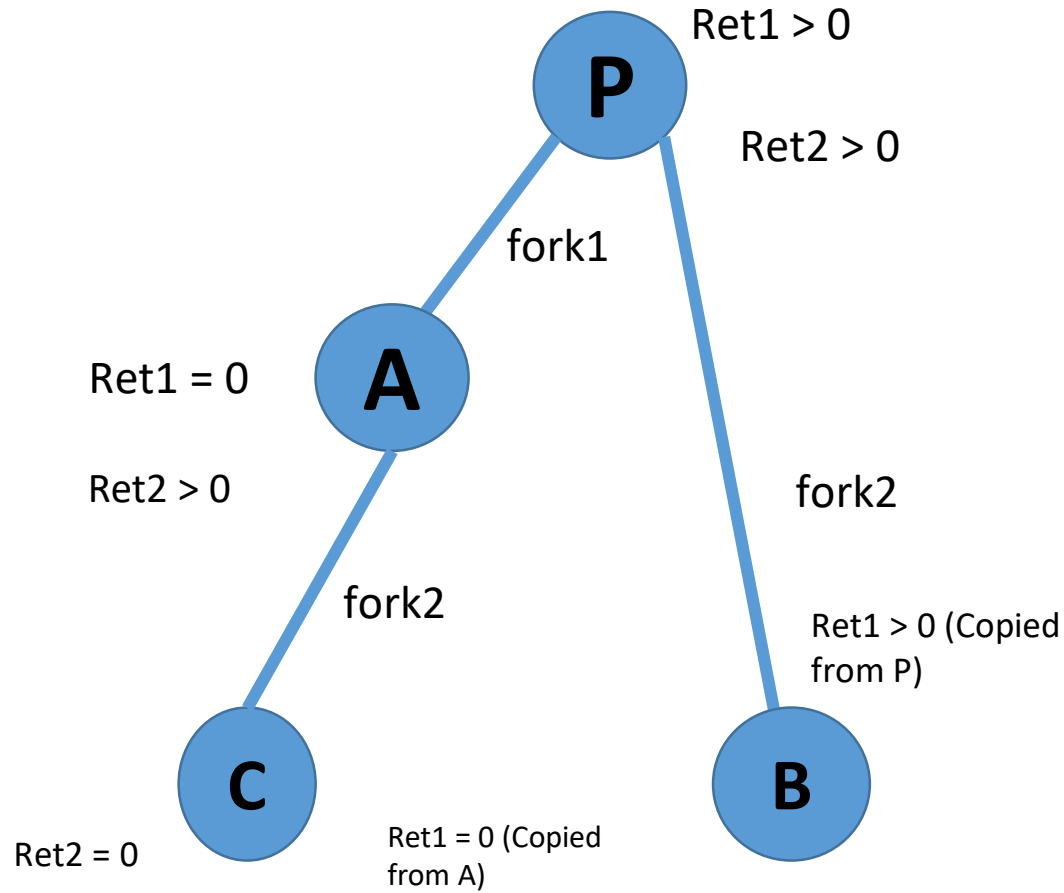
```

Process A

```

1.  pid_t ret1 = fork();
2.  pid_t ret2 = fork();
3.
4.  if(ret2 == 0)
5.  {
6.      printf("OS is the worst!\n");
7.  }else if(ret1 == 0 && ret2 > 0)
8.  {
9.      wait(NULL);
10.     printf("OS is the okay!\n");
11. }else if(ret1 > 0 && ret2 == 0)
12. {
13.     wait(NULL);
14.     printf("OS is fun!\n");
15. }else if(ret1 > 0 && ret2 > 0)
16. {
17.     int status;
18.     waitpid(ret2, &status, 0);
19.     printf("OS is confusing!\n");
20. }else
21. {
22.     wait(NULL);
23.     printf("OS is the best!\n");
24. }

```



Say, Process A is running now

Process C

```

1.  pid_t ret1 = fork();
2.  pid_t ret2 = fork();
3.
4.  if(ret2 == 0)
5.  {
6.      printf("OS is the worst!\n");
7.  }else if(ret1 == 0 && ret2 > 0)
8.  {
9.      wait(NULL);
10.     printf("OS is the okay!\n");
11. }else if(ret1 > 0 && ret2 == 0)
12. {
13.     wait(NULL);
14.     printf("OS is fun!\n");
15. }else if(ret1 > 0 && ret2 > 0)
16. {
17.     int status;
18.     waitpid(ret2, &status, 0);
19.     printf("OS is confusing!\n");
20. }else
21. {
22.     wait(NULL);
23.     printf("OS is the best!\n");
24. }

```

Process P

```

1. pid_t ret1 = fork();
2. pid_t ret2 = fork();
3.
4. if(ret2 == 0)
5. {
6.     printf("OS is the worst!\n");
7. }else if(ret1 == 0 && ret2 > 0)
8. {
9.     wait(NULL);
10.    printf("OS is the okay!\n");
11. }else if(ret1 > 0 && ret2 == 0)
12. {
13.     wait(NULL);
14.     printf("OS is fun!\n");
15. }else if(ret1 > 0 && ret2 > 0)
16. {
17.     int status;
18.     waitpid(ret2, &status, 0);
19.     printf("OS is confusing!\n");
20. }else
21. {
22.     wait(NULL);
23.     printf("OS is the best!\n");
24. }
```

Process A

```

1. pid_t ret1 = fork();
2. pid_t ret2 = fork();
3.
4. if(ret2 == 0)
5. {
6.     printf("OS is the worst!\n");
7. }else if(ret1 == 0 && ret2 > 0)
8. {
9.     wait(NULL);
10.    printf("OS is the okay!\n");
11. }else if(ret1 > 0 && ret2 == 0)
12. {
13.     wait(NULL);
14.     printf("OS is fun!\n");
15. }else if(ret1 > 0 && ret2 > 0)
16. {
17.     int status;
18.     waitpid(ret2, &status, 0);
19.     printf("OS is confusing!\n");
20. }else
21. {
22.     wait(NULL);
23.     printf("OS is the best!\n");
24. }
```

Process B

```

1. pid_t ret1 = fork();
2. pid_t ret2 = fork();
3.
4. if(ret2 == 0)
5. {
6.     printf("OS is the worst!\n");
7. }else if(ret1 == 0 && ret2 > 0)
8. {
9.     wait(NULL);
10.    printf("OS is the okay!\n");
11. }else if(ret1 > 0 && ret2 == 0)
12. {
13.     wait(NULL);
14.     printf("OS is fun!\n");
15. }else if(ret1 > 0 && ret2 > 0)
16. {
17.     int status;
18.     waitpid(ret2, &status, 0);
19.     printf("OS is confusing!\n");
20. }else
21. {
22.     wait(NULL);
23.     printf("OS is the best!\n");
24. }
```

Process C

```

1. pid_t ret1 = fork();
2. pid_t ret2 = fork();
3.
4. if(ret2 == 0)
5. {
6.     printf("OS is the worst!\n");
7. }else if(ret1 == 0 && ret2 > 0)
8. {
9.     wait(NULL);
10.    printf("OS is the okay!\n");
11. }else if(ret1 > 0 && ret2 == 0)
12. {
13.     wait(NULL);
14.     printf("OS is fun!\n");
15. }else if(ret1 > 0 && ret2 > 0)
16. {
17.     int status;
18.     waitpid(ret2, &status, 0);
19.     printf("OS is confusing!\n");
20. }else
21. {
22.     wait(NULL);
23.     printf("OS is the best!\n");
24. }
```

Now the system has 4 processes. And each of them will execute from line 3

Process P

```
1.  pid_t ret1 = fork();
2.  pid_t ret2 = fork();
3.
4.  if(ret2 == 0)
5.  {
6.      printf("OS is the worst!\n");
7.  }else if(ret1 == 0 && ret2 > 0)
8.  {
9.      wait(NULL);
10.     printf("OS is the okay!\n");
11. }else if(ret1 > 0 && ret2 == 0)
12. {
13.     wait(NULL);
14.     printf("OS is fun!\n");
15. }else if(ret1 > 0 && ret2 > 0)
16. {
17.     wait(NULL);
18.     printf("OS is confusing!\n");
19. }else
20. {
21.     wait(NULL);
22.     printf("OS is the best!\n");
23. }
```

For Process P,
Ret1 > 0
Ret2 > 0

This condition is true in
process P

Process P goes to waiting for
a child (B or A) according to
line 17.

Process A, B and C are in Ready
State, P in waiting state

Say, Process P runs first

Process A

```
1. pid_t ret1 = fork();
2. pid_t ret2 = fork();
3.
4. if(ret2 == 0)
5. {
6.     printf("OS is the worst!\n");
7. }else if(ret1 == 0 && ret2 > 0)
8. {
9.     wait(NULL);
10.    printf("OS is the okay!\n");
11. }else if(ret1 > 0 && ret2 == 0)
12. {
13.     wait(NULL);
14.     printf("OS is fun!\n");
15. }else if(ret1 > 0 && ret2 > 0)
16. {
17.     wait(NULL);
18.     printf("OS is confusing!\n");
19. }else
20. {
21.     wait(NULL);
22.     printf("OS is the best!\n");
23. }
```

For Process A,
Ret1 = 0
Ret2 > 0

This condition is true in
process A

Process A goes to waiting for
a child (Child C) according to
line 8.

Process B and C are in Ready
State, Process P and A are in
waiting/blocked State

Say, Process A runs next

Process B

```
1. pid_t ret1 = fork();
2. pid_t ret2 = fork();
3.
4. if(ret2 == 0)
5. {
6.     printf("OS is the worst!\n");
7. }
8. else if(ret1 == 0 && ret2 > 0)
9. {
10.    wait(NULL);
11.    printf("OS is the okay!\n");
12. }
13. else if(ret1 > 0 && ret2 == 0)
14. {
15.    wait(NULL);
16.    printf("OS is fun!\n");
17. }
18. else if(ret1 > 0 && ret2 > 0)
19. {
20.    wait(NULL);
21.    printf("OS is confusing!\n");
22. }
23. }
```

For Process B,
Ret1 > 0
Ret2 = 0

This condition is true in
process B

Process B Prints line 5

Output:

Os is the worst!

Now, Process P goes to Ready
State. Process C is in ready state.
Process A still in waiting

Say, Process B runs next

Process C

```
1. pid_t ret1 = fork();
2. pid_t ret2 = fork();
3.
4. if(ret2 == 0)
5. {
6.     printf("OS is the worst!\n");
7. }
8. else if(ret1 == 0 && ret2 > 0)
9. {
10.    wait(NULL);
11.    printf("OS is the okay!\n");
12. }
13. else if(ret1 > 0 && ret2 == 0)
14. {
15.    wait(NULL);
16.    printf("OS is fun!\n");
17. }
18. else if(ret1 > 0 && ret2 > 0)
19. {
20.    wait(NULL);
21.    printf("OS is confusing!\n");
22. }
23. }
```

For Process C,
Ret1 = 0
Ret2 = 0

This condition is true in
process C

Process C Prints line 5

Output:

Os is the worst!
Os is the worst!

Now, Process A goes to Ready
State. Process P is in ready state.

Say, Process C runs next

Process P

```
1. pid_t ret1 = fork();
2. pid_t ret2 = fork();
3.
4. if(ret2 == 0)
5. {
6.     printf("OS is the worst!\n");
7. }else if(ret1 == 0 && ret2 > 0)
8. {
9.     wait(NULL);
10.    printf("OS is the okay!\n");
11. }else if(ret1 > 0 && ret2 == 0)
12. {
13.     wait(NULL);
14.     printf("OS is fun!\n");
15. }else if(ret1 > 0 && ret2 > 0)
16. {
17.     wait(NULL);
18.     printf("OS is confusing!\n");
19. }else
20. {
21.     wait(NULL);
22.     printf("OS is the best!\n");
23. }
```

For Process P,
Ret1 > 0
Ret2 > 0

This condition is true in
process P

Process P now executes line
18

Output:

Os is the worst!
Os is the worst!
Os is confusing!

Process A is in ready state

Say, Process P runs next

Process A

```
1. pid_t ret1 = fork();
2. pid_t ret2 = fork();
3.
4. if(ret2 == 0)
5. {
6.     printf("OS is the worst!\n");
7. }else if(ret1 == 0 && ret2 > 0)
8. {
9.     wait(NULL);
10.    printf("OS is the okay!\n");
11. }else if(ret1 > 0 && ret2 == 0)
12. {
13.     wait(NULL);
14.     printf("OS is fun!\n");
15. }else if(ret1 > 0 && ret2 > 0)
16. {
17.     wait(NULL);
18.     printf("OS is confusing!\n");
19. }else
20. {
21.     wait(NULL);
22.     printf("OS is the best!\n");
23. }
```

For Process A,
Ret1 = 0
Ret2 > 0

This condition is true in
process A

Process A now executes line 9

Output:

Os is the worst!
Os is the worst!
Os is confusing!
Os is okay!

No process left

Say, Process A runs next